Lab 5 — Issues Found & Brief Explanation

NAME: Ibrahim khaleel
SRN: PES2UG25CS809
SECTION: D

Summary

Metric	Before	After
Pylint score	5.73 / 10	**10.00 / 10**
Bandit issues	11 (1 High, 5 Medium, 4 Low, 1 other)	**0**
Flake8 warnings	19	**0**

1. Issue Table

#	Issue	Type	Line(s)	Description	Fix Approach
1	Unsafe tar extraction (path traversal / Zip Slip)	Security — **Bandit B202 (HIGH)**	128–137	`tar.extractall()` extracted every archive member without validating paths, letting a malicious archive write outside `claims_docs` (e.g. via `../../etc/passwd`).	Resolve each member path with `os.path.realpath`, reject anything outside `claims_docs`, extract members individually.
2	Unverified SSL context	Security — **Bandit B323 (MEDIUM)**	116	`ssl._create_unverified_context()` disabled HTTPS certificate validation, exposing downloads to man-in-the-middle attacks.	Replaced with `ssl.create_default_context()`.
3	Unvalidated XML parsing	Security — **Bandit B314 / B405 (MEDIUM)**	1, 88	`xml.etree.ElementTree.fromstring` is vulnerable to XML bombs (billion laughs) and XXE.	Switched to `defusedxml.ElementTree`, a drop-in safe replacement.
4	Bind to all interfaces	Security — **Bandit B104 (MEDIUM)**	28	`HOST = "0.0.0.0"` binds to every network interface.	Changed to `"127.0.0.1"` (loopback only).
5	Hardcoded temp directory	Security — **Bandit B108 (MEDIUM)**	34	Log file hardcoded to `/tmp/premium_quotes.log`, a world-writable, predictable path.	Use `tempfile.gettempdir()`.
6	Hardcoded credential	Security — **Bandit B105 (LOW)**	31	Underwriter token hardcoded in source, so anyone with the repo has the secret.	Read from `os.environ["UNDERWRITER_TOKEN"]`.
7	Weak random for policy numbers	Security — **Bandit B311 (LOW)**	68	`random.randint` is not cryptographically secure; policy numbers are predictable.	Use `secrets.randbelow(900000)`.

#	Issue	Type	Line(s)	Description	Fix Approach
8	Bare `except:`	Bug / Style — **Bandit B112, Flake8 E722**	96	`except:` swallowed every error (including `KeyboardInterrupt`) and silently skipped malformed policies.	Catch specific `(ValueError, TypeError, AttributeError)`.
9	Mutable default argument	Bug — **Pylint W0102**	40	`discounts=[]` was shared across calls and then mutated with `discounts.append(500)`, corrupting later quotes.	Default to `None`, initialize inside; removed the stray `append`.
10	Negative premium possible	Logic Bug (not tool-flagged)	40–55	Stacked discounts could push premium below zero.	Added `return max(premium, 0)`.
11	Non-smoker charged smoker surcharge	Logic Bug (not tool-flagged)	main()	Passing the string `"no"` (truthy) applied the 1.3× smoker multiplier.	Passed `False`.
12	Age off-by-one	Logic Bug (not tool-flagged)	59–68	`ageFromDob` used only year subtraction, ignoring whether the birthday had passed this year.	Compare `(today.month, today.day)` to `(born.month, born.day)` and subtract 1 if needed.
13	Feb 29 renewal crash	Logic Bug (not tool-flagged)	155–162	`date.replace(year+1)` raises `ValueError` on Feb 29 in non-leap years.	Clamp the day with `calendar.monthrange`.
14	Missing docstrings	Style — **Pylint C0114 / C0116**	many	No module or function docstrings.	Added a module docstring and one-line docstrings to every function.
15	`camelCase` function names	Style — **Pylint C0103**	many	Names like `calculatePremium`, `ageFromDob` violate PEP 8.	Renamed to `snake_case`.
16	`open()` without encoding	Style — **Pylint W1514**	76	Locale-default encoding can corrupt non-ASCII data.	Added `encoding="utf-8"`.
17	`%` string formatting	Style — **Pylint C0209**	78	Old-style formatting.	Replaced with an f-string.
18	Resources not managed with `with`	Style — **Pylint R1732**	76, 130	Manual `open`/`close` and `tarfile.open`/`close` risk leaking handles.	Wrapped in `with` statements.
19	Unnecessary `else` after `return`	Style — **Pylint R1705**	143, 152	Redundant `else` blocks.	Removed `else`, de-indented.
20	Verbose boolean return	Style — **Pylint R1703**	149–152	`if x: return True else: return False`.	Replaced with a single `return bool(...) and ...` statement.
21	`range(len(...))` loop	Style — **Pylint C0200**	169	Manual index loop.	Replaced with `enumerate(...)`.
22	Lines over 79 chars	Style — **Flake8 E501 /**	40, 100, 133	Several lines exceed PEP 8.	Wrapped the offending lines.

#	Issue	Type	Line(s)	Description	Fix Approach
		Pylint C0301			
23	Missing blank lines between functions	Style — **Flake8 E302**	many	Only one blank line between top-level definitions.	Added a second blank line.
24	Missing `if __name__ == "__main__":` guard	Style / best practice	end	Demo code ran on import.	Added the guard.

2. Brief Explanation of Each Category

High-Severity Security

Issue #1 — Unsafe tar extraction. `tar.extractall()` is one of the most dangerous one-liners in Python. If the archive contains entries like `../../etc/passwd`, the file gets written outside the target folder. Attackers have used this ("Zip Slip") to overwrite config files, SSH keys, and cron jobs on production servers. The fix is to resolve every member's path and refuse anything outside `claims_docs`.

Medium-Severity Security

#2 — Unverified SSL. `ssl._create_unverified_context()` turns HTTPS into HTTP — the padlock lies. Anyone on the network can substitute a fake rate table. **#3 — XML.** Default Python XML parsers expand entities recursively, so a small XML document can exhaust memory ("billion laughs") or read local files (XXE). **#4 — `0.0.0.0`.** A service bound to all interfaces is reachable from every network the machine touches, not just localhost. **#5 — `/tmp`.** Predictable paths in a world-writable directory allow symlink attacks.

Low-Severity Security

#6 — Hardcoded token. Once committed, it's in Git history forever; rotate and load from the environment. **#7 — Weak random.** `random` is a PRNG seeded from the clock — an attacker can predict policy numbers. `secrets` uses the OS CSPRNG. **#8 — Bare `except`.** Silently swallows everything, including bugs you want to know about. It hides malformed input instead of surfacing it.

Logic Bugs (None of the tools found these)

These are the ones the lab README warned about. **#9 — Mutable default.** `discounts=[]` is a single list object shared by every call — that's why Quote 1 and Quote 2 printed different premiums despite identical inputs. **#10 — Negative premium.** Discounts summing to more than the premium produced a negative quote. **#11 — Truthy `"no"`.** In Python, `bool("no")` is `True`, so a non-smoker got the smoker surcharge. **#12 — Age off-by-one.** Birthday-aware age is not the same as `year_now - year_born`. **#13 — Feb 29.** `date.replace(year=2025)` from `2024-02-29` raises `ValueError` because 2025 is not a leap year.

Static analysis cannot know what a correct insurance premium is — it has no model of the domain. This is why tools complement, but do not replace, human review and tests.

Style & Maintainability

Docstrings, PEP 8 naming, `with` statements, f-strings, `enumerate`, encoding on `open`, blank-line spacing, and the `if __name__ == "__main__"` guard. Individually small; collectively they determine whether the next developer can read and safely modify the file. Flake8 enforces formatting mechanically; Pylint enforces structure; both keep the codebase consistent as it grows.

3. Why the Tools Overlap but Don't Duplicate

Tool	Catches	Misses
Pylint	Naming, docstrings, dangerous defaults, `with` usage, redundant code	Security vulnerabilities, line length <100
Flake8	PEP 8 formatting, blank lines, line length, bare `except`	Security, naming beyond PEP 8, design
Bandit	Unsafe APIs, TLS, XML, tar, hardcoded secrets, weak crypto	Style, docstrings, logic

None of the three caught the logic bugs (#10–#13). That gap is the central lesson of this lab: **static analysis is a safety net, not a substitute for reasoning about the output.**